2025 集创赛龙芯中科杯
分区赛决赛发布包说明手册

目 录

第 1 章 分区赛决赛前端环境 5
1.1 概述.....5
1.2 修复逻辑综合并添加 IOPAD.....5
1.3 添加中断控制器的步骤.....11
第 2 章 分区赛决赛 SoC 扩展 11
2.1 分区赛决赛 SoC 扩展任务.....11
2.1 实验流程.....11
2.1.1 补齐 Confreg 的中断控制器 11
2.1.2 运行 int_test 测试.....14
2.1.3 运行 RTThread_Nano 测试 17

图目录

图 2-1　int_test 测试程序运行仿真的串口输出 16
图 2-2　int_test 测试程序进行 FPGA 验证的串口输出.....16
图 2-3　系统测试程序运行仿真的串口输出 17
图 2-4　系统测试程序运行仿真的 LED 输出 17
图 2-5　系统测试程序的 FPGA 验证结果.....18
图 2-6　系统测试程序的 FPGA 串口输出内容.....18

表目录

表 1-1 iopad 调用宏 6
表 2-1 外部中断控制器信号列表 11
表 2-2 外部中断控制器寄存器总览 12

第 1 章　分区赛决赛前端环境

1.1　概述

分区赛决赛前端环境与初赛发布包目录一致，请参阅《集创赛龙芯中科杯初赛发布包说明》。分区赛决赛发布包主要为 SoC 添加外部中断控制器提供测试程序。外部中断控制器的设计需求详见本文第二章。

分区赛决赛发布包与初赛发布包的区别：

1. 硬件部分修复 DC 综合中的 ERROR，并添加 IOPAD。
2. 软件部分添加中断测试程序(sdk/software/examples/int_test)和系统测试程序(sdk/software/examples/RTThread_Nano)。用于测试 SoC 的中断功能是否正常。

下面介绍分区赛决赛发布包使用方法。

1.2　修复逻辑综合并添加 IOPAD

Step1：替换 RTL 文件

将下列分区赛决赛发布包中的文件替换至你的工作目录。

① rtl/ip/Bus_interconnects/axi2sram_sp_ext.v
② rtl/ip/confreg/confreg.v
③ rtl/ip/confreg/digitaltube_controller.v
④ rtl/ip/open-la500/csr.v
⑤ rtl/ip/open-la500/id_stage.v
⑥ rtl/ip/open-la500/if_stage.v
⑦ rtl/ip/open-la500/wb_stage.v
⑧ rtl/ip/open-la500/mycpu_top.v
⑨ rtl/ip/ram_wrap/axi_wrap_ram_sp_ext.v
⑩ rtl/config.h
⑪ rtl/iopad.svh
⑫ rtl/iopad_fpga.v

Step2：修改顶层文件

① 首先修改文件名，将 soc_top.v 改为 soc_top.sv，用于支持 system verilog 语法。

② 修改 interface，删除不需要的 IO，在最简设计中，应删除下列 IO:

```
output [2:0]    video_red,          //红色像素，3 位
output [2:0]    video_green,        //绿色像素，3 位
output [1:0]    video_blue,         //蓝色像素，2 位
```

```
output          video_hsync,        //行同步（水平同步）信号
output          video_vsync,        //场同步（垂直同步）信号
output          video_clk,          //像素时钟输出
output          video_de,           //行数据有效信号，用于区分消隐区
input           clock_btn,          //BTN5 手动时钟按钮开关，带消抖电路，按下时为1

output [22:0]   flash_a,            //Flash 地址，a0 仅在 8bit 模式有效，16bit 模式无意义
inout  [15:0]   flash_d,            //Flash 数据
output          flash_rp_n,         //Flash 复位信号，低有效
output          flash_vpen,         //Flash 写保护信号，低电平时不能擦除、烧写
output          flash_ce_n,         //Flash 片选信号，低有效
output          flash_oe_n,         //Flash 读使能信号，低有效
output          flash_we_n,         //Flash 写使能信号，低有效
output          flash_byte_n,       //Flash 8bit 模式选择，低有效。在使用 flash 的 16 位模式时请设为 1
```

若设计中使用了上述 IO 信号，可以保留，并注意在第④步中自行添加 iopad。

另外还需要添加一个 output 信号 clk_o，因为在 ASIC 设计中使用的是无源晶振，有 XTALI 和 XTALO 两个信号，而 FPGA 上是有源晶振只需要 XTALI。

```
output          clk_o,              //XTALO
```

③ 修改 xdc 文件

根据删除的信号，删除 fpga/constrains/soc.xdc 中对应的 IO 约束，并添加 clk_o 的 IO 约束，将 clk_o 约束到一个不用的 FPGA 引脚上,添加的内容如下。

```
set_property -dict {IOSTANDARD LVCMOS33 PACKAGE_PIN J26} [get_ports clk_o]
```

④ 添加 iopad

在 ASIC 设计中，一般会调用工艺库提供的 iopad 单元实现 IO 与内部 module 的连接。在分区赛决赛发布包中，rtl/iopad.svh 文件提供调用 iopad 单元的宏定义，在 soc_top.sv 文件中添加`include "iopad.svh"。下表给出针对不同的 IO 对应需要调用的宏。

表 1-1 iopad 调用宏

信号类型	宏
单比特 input	IPAD_GEN_SIMPLE
多比特 input	IPAD_GEN_VEC_SIMPLE
单比特 output	OPAD_GEN_SIMPLE

多比特 output	OPAD_GEN_VEC_SIMPLE
单比特 inout	IOPAD_GEN_SIMPLE
多比特 inout	IOPAD_GEN_VEC_SIMPLE

在最简设计中，应该在 soc_top.sv 中增加下述代码以添加 iopad。

```
PX3W PAD_CLK_IN (.XIN(clk), .XOUT(clk_o), .XC(clk_i));
`IPADU_GEN_SIMPLE(reset)
`IPAD_GEN_VEC_SIMPLE(touch_btn)
`IPAD_GEN_VEC_SIMPLE(dip_sw)
`OPAD_GEN_VEC_SIMPLE(leds)
`OPAD_GEN_VEC_SIMPLE(dpy0)
`OPAD_GEN_VEC_SIMPLE(dpy1)
`IOPAD_GEN_VEC_SIMPLE(base_ram_data)
`OPAD_GEN_VEC_SIMPLE(base_ram_addr)
`OPAD_GEN_VEC_SIMPLE(base_ram_be_n)
`OPAD_GEN_SIMPLE(base_ram_ce_n)
`OPAD_GEN_SIMPLE(base_ram_oe_n)
`OPAD_GEN_SIMPLE(base_ram_we_n)
`IOPAD_GEN_VEC_SIMPLE(ext_ram_data)
`OPAD_GEN_VEC_SIMPLE(ext_ram_addr)
`OPAD_GEN_VEC_SIMPLE(ext_ram_be_n)
`OPAD_GEN_SIMPLE(ext_ram_ce_n)
`OPAD_GEN_SIMPLE(ext_ram_oe_n)
`OPAD_GEN_SIMPLE(ext_ram_we_n)
`IOPAD_GEN_SIMPLE(UART_RX)
`IOPAD_GEN_SIMPLE(UART_TX)
```

其中 PX3W 是给 input clk 专用的 iopad，需要自行添加一个信号定义 wire clk_i。IPADU_GEN_SIMPLE 是针对扇出较大的 input reset 信号的宏。如果参赛设计使用了其他 IO，请参考上述代码自行添加对应的 iopad。这些宏将调用工艺库提供的 iopad 模块，在 FPGA 上验证时，使用 iopad_fpga.v 实现这些 iopad 的功能。

⑤ 修改模块实例化时连接的 io 信号名

添加了 iopad 后，SoC 内部的 module 能看到的信号不再是之前的 touch_btn、leds、base_ram_data，而是 touch_btn_i、leds_o、base_ram_data_i、base_ram_data_o、base_ram_data_oe。即 input 信号多了后缀_i，output 信号多了后缀_o，inout 信号的对内接口变为三个信号，分别是输入信号_i，输出信号_o，输出使能信号_oe（低电平有效，为 0 时该 iopad 输出，为 1 时输入）。由于对内信号名的变化，soc_top.sv 中对各个模块实例化时连接的信号名也需要跟着修改，下面给出 SRAM 控制器和串口控制器的实例化示例，其余有连接 IO 的模块请自行修改信号名，尤其注意修改 clk_i 和 reset_i。

SRAM 控制器：

```
//axi ram
axi_wrap_ram_sp_ext u_axi_ram (
    .aclk          ( sys_clk          ),
    .aresetn       ( sys_resetn       ),
    //ar
    .axi_arid      ( ram_arid         ),
    .axi_araddr    ( ram_araddr       ),
    .axi_arlen     ( ram_arlen        ),
    .axi_arsize    ( ram_arsize       ),
    .axi_arburst   ( ram_arburst      ),
    .axi_arlock    ( ram_arlock       ),
    .axi_arcache   ( ram_arcache      ),
    .axi_arprot    ( ram_arprot       ),
    .axi_arvalid   ( ram_arvalid      ),
    .axi_arready   ( ram_arready      ),
    //r
    .axi_rid       ( ram_rid          ),
    .axi_rdata     ( ram_rdata        ),
    .axi_rresp     ( ram_rresp        ),
    .axi_rlast     ( ram_rlast        ),
    .axi_rvalid    ( ram_rvalid       ),
    .axi_rready    ( ram_rready       ),
    //aw
    .axi_awid      ( ram_awid         ),
    .axi_awaddr    ( ram_awaddr       ),
    .axi_awlen     ( ram_awlen        ),
    .axi_awsize    ( ram_awsize       ),
    .axi_awburst   ( ram_awburst      ),
    .axi_awlock    ( ram_awlock       ),
    .axi_awcache   ( ram_awcache      ),
    .axi_awprot    ( ram_awprot       ),
    .axi_awvalid   ( ram_awvalid      ),
    .axi_awready   ( ram_awready      ),
    //w
    .axi_wdata     ( ram_wdata        ),
    .axi_wstrb     ( ram_wstrb        ),
    .axi_wlast     ( ram_wlast        ),
    .axi_wvalid    ( ram_wvalid       ),
    .axi_wready    ( ram_wready       ),
    //b             ram
    .axi_bid       ( ram_bid          ),
    .axi_bresp     ( ram_bresp        ),
    .axi_bvalid    ( ram_bvalid       ),
    .axi_bready    ( ram_bready       ),

    .base_ram_addr ( base_ram_addr_o      ),
    .base_ram_be_n ( base_ram_be_n_o      ),
    .base_ram_ce_n ( base_ram_ce_n_o      ),
    .base_ram_oe_n ( base_ram_oe_n_o      ),
    .base_ram_we_n ( base_ram_we_n_o      ),
    .ext_ram_addr  ( ext_ram_addr_o       ),
    .ext_ram_be_n  ( ext_ram_be_n_o       ),
    .ext_ram_ce_n  ( ext_ram_ce_n_o       ),
    .ext_ram_oe_n  ( ext_ram_oe_n_o       ),
    .ext_ram_we_n  ( ext_ram_we_n_o       ),

    .base_ram_data_i    ( base_ram_data_i      ),
    .base_ram_data_o    ( base_ram_data_o      ),
    .base_ram_data_oe   ( base_ram_data_oe     ),
    .ext_ram_data_i     ( ext_ram_data_i       ),
    .ext_ram_data_o     ( ext_ram_data_o       ),
    .ext_ram_data_oe    ( ext_ram_data_oe      )

);
```

串口控制器：

```
//AXI2APB
axi_uart_controller u_axi_uart_controller
(
    .clk              (sys_clk              ),
    .rst_n            (sys_resetn           ),

    .axi_s_awid       (uart_awid            ),
    .axi_s_awaddr     (uart_awaddr          ),
    .axi_s_awlen      (uart_awlen           ),
    .axi_s_awsize     (uart_awsize          ),
    .axi_s_awburst    (uart_awburst         ),
    .axi_s_awlock     (uart_awlock          ),
    .axi_s_awcache    (uart_awcache         ),
    .axi_s_awprot     (uart_awprot          ),
    .axi_s_awvalid    (uart_awvalid         ),
    .axi_s_awready    (uart_awready         ),
    .axi_s_wid        (uart_wid             ),
    .axi_s_wdata      (uart_wdata           ),
    .axi_s_wstrb      (uart_wstrb           ),
    .axi_s_wlast      (uart_wlast           ),
    .axi_s_wvalid     (uart_wvalid          ),
    .axi_s_wready     (uart_wready          ),
    .axi_s_bid        (uart_bid             ),
    .axi_s_bresp      (uart_bresp           ),
    .axi_s_bvalid     (uart_bvalid          ),
    .axi_s_bready     (uart_bready          ),
    .axi_s_arid       (uart_arid            ),
    .axi_s_araddr     (uart_araddr          ),
    .axi_s_arlen      (uart_arlen           ),
    .axi_s_arsize     (uart_arsize          ),
    .axi_s_arburst    (uart_arburst         ),
    .axi_s_arlock     (uart_arlock          ),
    .axi_s_arcache    (uart_arcache         ),
    .axi_s_arprot     (uart_arprot          ),
    .axi_s_arvalid    (uart_arvalid         ),
    .axi_s_arready    (uart_arready         ),
    .axi_s_rid        (uart_rid             ),
    .axi_s_rdata      (uart_rdata           ),
    .axi_s_rresp      (uart_rresp           ),
    .axi_s_rlast      (uart_rlast           ),
    .axi_s_rvalid     (uart_rvalid          ),
    .axi_s_rready     (uart_rready          ),

    .apb_rw_dma       (1'b0                 ),
    .apb_psel_dma     (1'b0                 ),
    .apb_enab_dma     (1'b0                 ),
    .apb_addr_dma     (20'b0                ),
    .apb_valid_dma    (1'b0                 ),
    .apb_wdata_dma    (32'b0                ),
    .apb_rdata_dma    (                     ),
    .apb_ready_dma    (                     ),
    .dma_grant        (                     ),

    .dma_req_o        (                     ),
    .dma_ack_i        (1'b0                 ),

    //UART0
    .uart0_txd_i      (UART_TX_i            ),
    .uart0_txd_o      (UART_TX_o            ),
    .uart0_txd_oe     (UART_TX_oe           ),
    .uart0_rxd_i      (UART_RX_i            ),
    .uart0_rxd_o      (UART_RX_o            ),
    .uart0_rxd_oe     (UART_RX_oe           ),
    .uart0_rts_o      (uart0_rts_o          ),
    .uart0_dtr_o      (uart0_dtr_o          ),
    .uart0_cts_i      (uart0_cts_i          ),
```

```
    .uart0_dsr_i        (uart0_dsr_i            ),
    .uart0_dcd_i        (uart0_dcd_i            ),
    .uart0_ri_i         (uart0_ri_i             ),
    .uart0_int          (uart0_int              )
);
```

⑥ 修改 testbench 文件（sim/mycpu_tb.v）中顶层的实例化，删除不用的 IO，并添加 clk_o。

```
wire clk_o;
soc_top  #(.SIMULATION(1'b1)) u_soc_top (
    .clk                    ( clk           ),
    .reset                  ( reset         ),
    .clk_o                  ( clk_o         ),
    .touch_btn              ( touch_btn     ),
    .dip_sw                 ( dip_sw        ),
    .leds                   ( leds          ),
    .dpy0                   ( dpy0          ),
    .dpy1                   ( dpy1          ),

    .base_ram_addr          ( base_ram_addr   ),
    .base_ram_be_n          ( base_ram_be_n   ),
    .base_ram_ce_n          ( base_ram_ce_n   ),
    .base_ram_oe_n          ( base_ram_oe_n   ),
    .base_ram_we_n          ( base_ram_we_n   ),
    .ext_ram_addr           ( ext_ram_addr    ),
    .ext_ram_be_n           ( ext_ram_be_n    ),
    .ext_ram_ce_n           ( ext_ram_ce_n    ),
    .ext_ram_oe_n           ( ext_ram_oe_n    ),
    .ext_ram_we_n           ( ext_ram_we_n    ),

    .base_ram_data          ( base_ram_data   ),
    .ext_ram_data           ( ext_ram_data    ),

    .UART_RX                ( UART_RX       ),
    .UART_TX                ( UART_TX       )
);
```

⑦ 完成修改，试试之前初赛跑过的 ciciec_func 是否仿真和 FPGA 验证依然正常。

⑧ 在进行 DC 逻辑综合时，还需手动调整一下顶层文件。此时，iopad 由工艺库提供，不需要使用 iopad_fpga.v。并且不使用 pll，请将时钟和复位相关逻辑用下述代码替换。CDC 模块不需要删除。并关闭 cache，将 rtl/config.h 中的宏 `define USE_CACHE 注释掉。这样处理完后，不需要使用 pll 和 SRAM 这些 hard macro，物理设计会更简单。

```
assign cpu_clk = clk_i;
assign sys_clk = clk_i;
rst_sync u_rst_sys(
    .clk(sys_clk),
    .rst_n_in(~reset_i),
    .rst_n_out(sys_resetn)
);
rst_sync u_rst_cpu(
    .clk(cpu_clk),
    .rst_n_in(sys_resetn),
    .rst_n_out(cpu_resetn)
);
```

1.3 添加中断控制器的步骤

Step1: 将分区赛决赛发布包中的 sdk/software/examples/int_test 文件夹和 sdk/software/examples/RTThread_Nano 文件夹拷贝至初赛在 WSL Ubuntu 中保存的初赛发布包 sdk/software/examples 目录下。

Step2: 根据本文第二章的任务为 SoC 添加外部中断控制器。

Step3: 先编译 sdk/software/examples/int_test 测试程序（程序逻辑简单，适合调试,程序介绍在 2.1.2 节中给出），在 vivado 中进行功能仿真，通过后进行 FPGA 验证。

Step4: 编译系统测试程序 sdk/software/examples/RTThread_Nano，在 vivado 中进行功能仿真，通过后进行 FPGA 验证。

第 2 章　分区赛决赛 SoC 扩展

2.1 分区赛决赛 SoC 扩展任务

发布包的基础 Confreg 模块中，有输入的 4bit 按键信号 touch_btn[3:0]（可以视为上升沿触发的中断信号）和定时器产生的高电平触发的中断信号 timer_int。为了统一管理这些外部中断信号，SoC 中需要添加外部中断控制器。外部中断控制器仅输出 1bit 的中断信号给到处理器核的 HWI0（即 OpenLA500 的 intrpt[0]）。外部中断控制器支持中断使能控制、电平中断/脉冲中断选择，中断电平设置、中断清除、中断置位、中断状态查询。

完成添加中断控制器的电路应能通过系统测试程序 RTThread_Nano，在功能仿真和 FPGA 验证中输出正确结果。

2.1 实验流程

2.1.1 补齐 Confreg 的中断控制器

外部中断控制器可以接收来自各类外设的中断信号输入，支持高电平/低电平/上升沿/下降沿触发，输出的中断信号 int_out 为高电平信号，将送至处理器核的 HWI0（即 OpenLA500 的 intrpt[0]）。

外部中断控制器的信号列表在下面给出，请在 confreg.v 中实例化该外部中断控制器。外部中断控制器的输入控制信号（int_en、int_edge 等）由外部中断控制器的寄存器提供，寄存器需自行添加，见表 2-2。外部中断控制器的输出信号 int_out 连接 confreg 模块的输出信号 confreg_int，之后 confreg_int 连接 OpenLA500 的 intrpt[0]。

表 2-1　外部中断控制器信号列表

信号	IO	描述
sys_clk	input	外部中断控制器工作时钟
sys_resetn	input	外部中断控制器输入复位信号，低有效
cpu_clk	input	用于跨时钟域处理，int_out 将输出给处理器核
cpu_resetn	input	用于跨时钟域处理，int_out 将输出给处理器核
int_en[31:0]	input	输入的 32 位中断使能，为 1 表示使能比特位对应设备中断
int_edga[31:0]	input	为 1 表示边沿触发中断,为 0 表示电平触发
int_pol[31:0]	input	当 int_edge 配置成电平触发时,为 1 表示高电平触发中断,为 0 表示低电平触发；当 int_edge 配置成边沿触发时,为 1 表示上升沿触发中断,为 0 表示下降沿触发
int_in[31:0]	input	输入的 32 位中断信号
int_clr[31:0]	input	为 1 表示清除中断（针对边沿触发的中断）
int_state[31:0]	output	为 1 表示对应位的中断有效
int_out	output	输出给处理器核的中断信号

其中输入中断 int_in[3:0]接按键 touch_btn_data，int_in[4]接外部定时器中断 timer_int。

外部中断控制器的寄存器列表如下，请在 confreg.v 中添加下述寄存器。

表 2-2　外部中断控制器寄存器总览

名称	地址	描述
confreg_int_en	1f20f000	中断使能寄存器，可读可写，为 1 表示使能比特位对应设备中断
confreg_int_edge	1f20f004	中断边沿选择寄存器，可读可写，为 1 表示边沿触发中断,为 0 表示电平触发
confreg_int_pol	1f20f008	中断极性选择寄存器，可读可写，当 int_edge 配置成电平触发时,为 1 表示高电平触发中断,为 0 表示低电平触发； 当 int_edge 配置成边沿触发时,为 1 表示上升沿触发中断,为 0 表示下降沿触发
confreg_int_clr	1f20f00c	中断清除寄存器，只写，为 1 表示清除中断（针对边沿触发的中断）
confreg_int_state	1f20f014	中断状态寄存器，只读，为 1 表示比特位对应设备产生中断。

对于中断控制器的输入控制信号，下面给出示例帮助理解。

示例一：假设目前需要处理一个上升沿触发的中断源，该中断源连接 int_in[0]，则 int_en[0]应置 1，int_edge[0]置 1，int_pol[0]置 1；当 int_in[0]检测到上升沿后，将 int_state[0]置 1，并且 int_out 输出经过跨时钟域处理的高电平 1。处理器核的 intrpt[0]检测到高电平的中断请求后，将进入中断服务函数进行处理，中断服务函数通过读取 int_state 的值判断具体中断源，由于是上升沿触发的中断，需要处理器将 int_clr[0]置 1 来清除中断。外部中断控制器收到 int_clr[0]信号置 1 时，需要将 int_state[0]置 0，若没有其他中断，则 int_out 也为 0。int_clr 信号仅被拉高一个时钟周期，即下一周期自动置 0。

示例二：假设目前需要处理一个高电平触发的中断源（如定时器），该中断源连接 int_in[0]，则 int_en[0]应置 1，int_edge[0]置 0，int_pol[0]置 1；当 int_in[0]检测到高电平后，将 int_state[0]置 1，并且 int_out 输出经过跨时钟域处理的高电平 1。处理器核的中断服务函数将直接写中断源（即定时器）的寄存器，使得 int_in[0]置 0，此时外部中断控制器也应将 int_state[0]置 0，若没有其他中断，则 int_out 也为 0。

注意边沿触发和电平触发中断的清除方案是不同的，边沿触发需要使用 int_clr 寄存器，而电平触发不需要。

2.1.2 运行 int_test 测试

初步验证外部中断控制器的功能，可以使用 int_test 测试程序，该测试软件主要用于检测外部中断控制器管理的按键中断和外部定时器中断是否能够正常进入中断服务函数并清中断。

测试软件将首先初始化处理器核中与中断相关的 CSR 寄存器。

```
/* init exception */
la.local t0, trap_handler
csrwr    t0, csr_eentry

#clear int
li.w       t1, 0x1
csrwr      t1, csr_ticlr

#enable int
li.w       t1, 0x4
csrxchg    t1, t1, csr_crmd
li.w       t1, 0x1fff
csrwr      zero, csr_ecfg
csrwr      t1, csr_ecfg
```

在 start.S 中的处理器核外部中断使能代码，首先初始化了 EENTRY，将函数 trap_handler 作为例外服务函数入口，之后向 TICLR 写 1 确保处理器核内的定时器中断已被清除，向 CRMD[2]（全局中断使能）写 1 使能中断，最后向 ECFG 写 0x1ffff 使能所有中断源。完成这部分初始化后，当外部中断控制器向处理器核的 HWI0 发中断申请时，PC 便会跳转至 trap_handler 函数（位于 trap_handler.S）。

```
trap_handler:

    TRAP_ENTRY_SETJMP

    csrrd   a0, csr_estat
    csrrd   a1, csr_era

    srli.w  t0, a0, 16
    bne     zero, t0, is_exceptions
    andi    t0, a0, 0x3FC
    srli.w  t1, t0, 2

    # 根据中断号跳转到相应的处理程序 priority
    move t2, t1
    andi t2, t2, 0x1
    li.w t8, 0x1
    beq  t2, t8, handle_hwi0

    move t2, t1
    andi t2, t2, 0x2
    li.w t8, 0x2
    beq  t2, t8, handle_hwi1

    ......

    b trap_jump_exit

is_exceptions:
    addi.w t8, a1, 4
    csrwr t8, csr_era
1:
    b 1b

```

```
34. trap_jump_exit:
35.     TRAP_EXIT_LONGJMP
36.
37.
38. # HWI0 处理程序
39. handle_hwi0:
40.
41.     bl HWI0_IntrHandler
42.
43.     b trap_jump_exit
```

在 trap_handler 函数中首先进行保存现场，将寄存器堆数据压栈，之后读取 CSR.ESTAT，根据 Ecode 和 EsubCode 的值判断是 exception 还是 interrupt，若是 exception 则跳转至图中第 28 行 is_exceptions，将 CSR.ERA 加 0x4 后陷入死循环。若为 interrupt 则判断硬中断号，跳转至对应的 handle_hwix。本次实验中将外部中断控制器输出的中断请求连接至 HWI0，因此触发外部中断后应掉转至 handle_hwi0 再跳转至 HWI0_IntrHandler。

在 main.c 中需要对外部中断控制器和外部定时器初始化，下面是初始化代码，通过配置寄存器完成，定时器中断设置为每 1ms 触发一次。

```
1. RegWrite(0xbf20f004,0x0f);//edge
2. RegWrite(0xbf20f008,0x1f);//pol
3. RegWrite(0xbf20f00c,0x1f);//clr
4. RegWrite(0xbf20f000,0x1f);//en
5.
6. RegWrite(0xbf20f104,0x50000);//timercmp 1ms
7. RegWrite(0xbf20f108,0x1);//timeren
```

进入 HWI0_IntrHandler 后的处理也在 main.c 中，首先读取外部中断控制器的 int_state 判断是外部定时器中断/按键中断，外部中断控制器输入中断的 bit3-bit0 是按键，bit4 是外部定时器，int_state 若为 0x10 则为外部定时器中断，若低四位中有高电平则是按键中断。每触发一次定时器中断会串口输出“timer int”，每触发一次按键中断会输出“buttonx int”。

```
1. void HWI0_IntrHandler(void)
2. {
3.     unsigned int int_state;
4.     int_state = RegRead(0xbf20f014);
5.
6.     if((int_state & 0x10) == 0x10){
7.         Timer_IntrHandler();
8.     }
9.     else if(int_state & 0xf){
10.        Button_IntrHandler(int_state & 0xf);
11.    }
12. }
13.
14. void Timer_IntrHandler(void)
15. {
16.     RegWrite(0xbf20f108,0);//timeren
17.     RegWrite(0xbf20f108,1);//timeren
18.     printf("timer int\n");
19. }
20.
21. void Button_IntrHandler(unsigned char button_state)
22. {
23.     if((button_state & 0b1000) == 0b1000){
```

```
        printf("button4 int\n");
        RegWrite(0xbf20f00c,0x8);//clr
    }
    else if((button_state & 0b0100) == 0b0100){
        printf("button3 int\n");
        RegWrite(0xbf20f00c,0x4);//clr
    }
    else if((button_state & 0b0010) == 0b0010){
        printf("button2 int\n");
        RegWrite(0xbf20f00c,0x2);//clr
    }
    else if((button_state & 0b0001) == 0b0001){
        printf("button1 int\n");
        RegWrite(0xbf20f00c,0x1);//clr
    }
}
```

在 vivado 中进行功能仿真，可以看到下图结果，四个按键中断和一个定时器中断的服务函数都能正确执行。

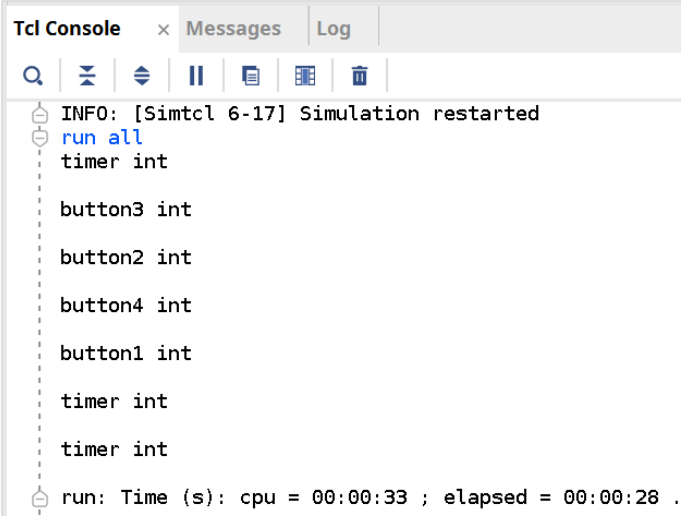

图 2-1　int_test 测试程序运行仿真的串口输出

进行 FPGA 验证，可以先看到 1 秒输出一次的“timer int”，手动点击 button 按键，可以看到串口输出“buttonx int”。

图 2-2　int_test 测试程序进行 FPGA 验证的串口输出

2.1.3 运行 RTThread_Nano 测试

下面使用 SoC 运行嵌入式实时操作系统：RTThread_Nano，程序位于 sdk/software/examples/RTThread_Nano 目录。该测试也是本次竞赛的系统测试程序。测试程序中添加了三个线程，分别是两个 LED 线程（led1、led）和一个命令行线程(tshell)。线程 led1 进行 LED1 和 LED2 的电平切换、现成 led2 进行 LED3 和 LED4 的电平切换。两个 LED 线程实现的效果是首先 LED1 和 LED3 先点亮，一秒后（仿真中是 1ms 后）LED2 和 LED4 点亮，再一秒后 LED1 和 LED3 熄灭……无限循环。命令行线程可以接收串口输入的命令并进行反馈。下图给出了 RT 测试程序在仿真中的串口输出内容，msh>代表 tshell 线程已经开始运行等待命令输入。

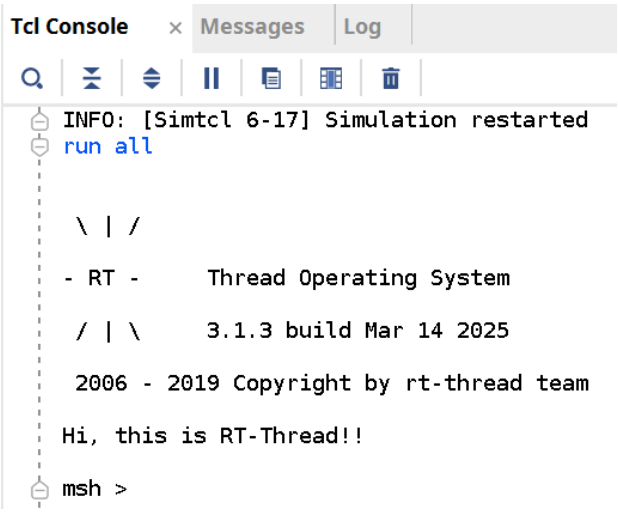

图 2-3　系统测试程序运行仿真的串口输出

下图是 RT 测试程序在仿真中的 LED 输出，LED 灯交替亮灭，代表两个 LED 线程运行正常。

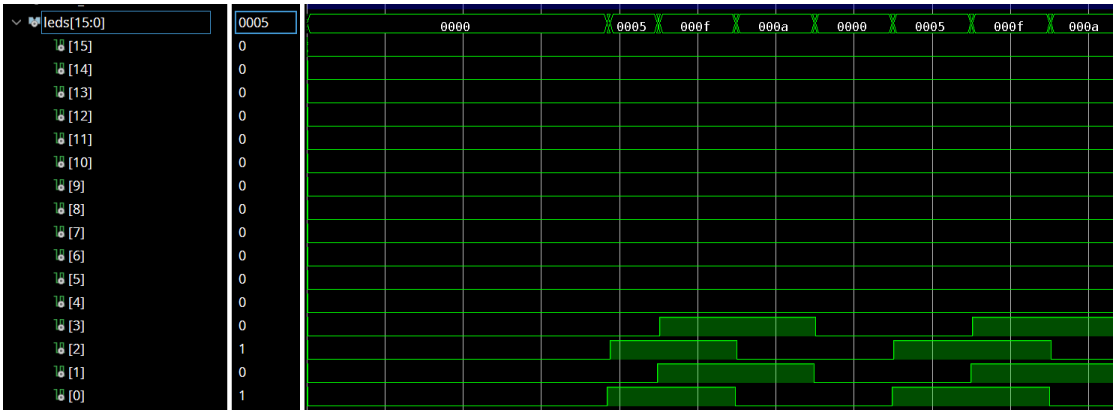

图 2-4　系统测试程序运行仿真的 LED 输出

进行 FPGA 验证时可以看到四个 LED 灯 D1、D3、D2、D4 交替亮灭，每一次电平跳变的间隔时间是 1 秒。右侧串口输出了 RTThread 的版本信息并且 msh 正在等待命令输出。

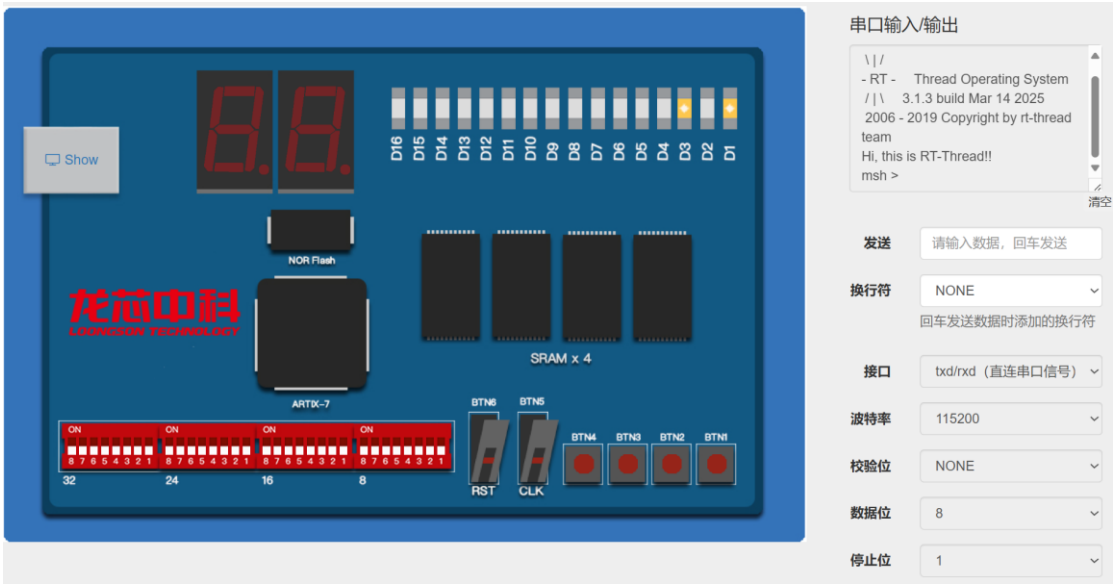

图 2-5　系统测试程序的 FPGA 验证结果

点击串口下方的`终端模式`可以进行命令行交互，按下 Tab 键可以输出所有支持的命令，执行 ps 命令可以列出当前所有线程的情况。下图给出 FPGA 执行 RT 测试程序的串口输出结果，从命令行的响应可以看出 RTThread 创建的三个线程和默认的 tidle 线程都正常执行。

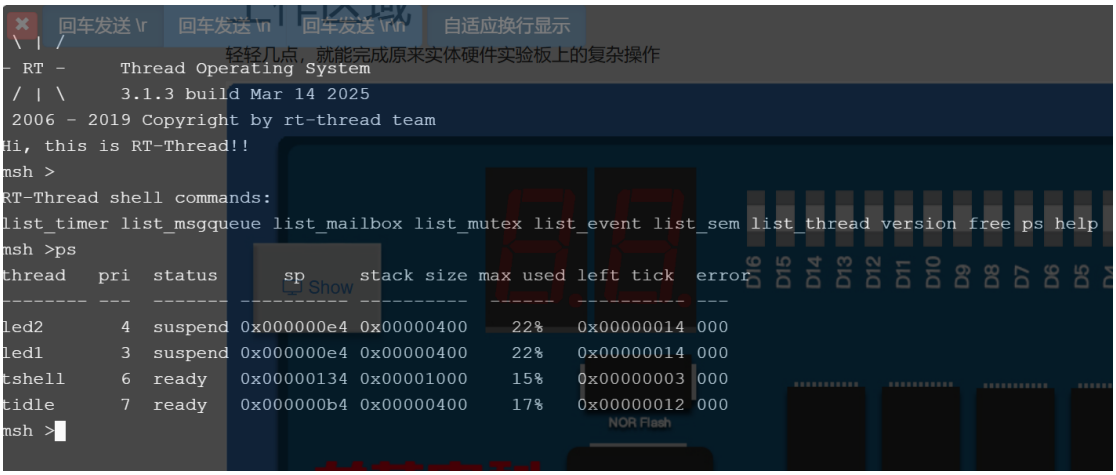

图 2-6　系统测试程序的 FPGA 串口输出内容